

```
import pandas as pd

!gdown 1yxgr0Qj3TiXRehYa0PED1t4zIga9gdY5
```

Downloading...
 From: <https://drive.google.com/uc?id=1yxgr0Qj3TiXRehYa0PED1t4zIga9gdY5>
 To: /content/churn_prediction.xlsx
 100% 494k/494k [00:00<00:00, 83.8MB/s]

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import roc_auc_score, accuracy_score, precision_score, recall_score
```

```
df = pd.read_excel("/content/churn_prediction.xlsx")
df.head(2)
```

	CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	Numt
0	52869	0	20.0	Mobile Phone	3	7.0	E wallet	Female	4.0	
1	52942	0	13.0	Computer	1	9.0	Cash on Delivery	Female	4.0	

```
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
import numpy as np
warnings.filterwarnings('ignore')

from sklearn.preprocessing import MinMaxScaler
```

1. Data Preprocessing:

```
# Change all column names to lowercase for easy to mention:
df.columns = df.columns.str.lower()
```

```
# Check imbalance data:
imb_df = df.groupby('churn')['customerid'].count().reset_index()
imb_df['%'] = imb_df['customerid']/sum(imb_df['customerid'])
imb_df
```

	churn	customerid	%
0	0	4682	0.831616
1	1	948	0.168384

```
df.shape

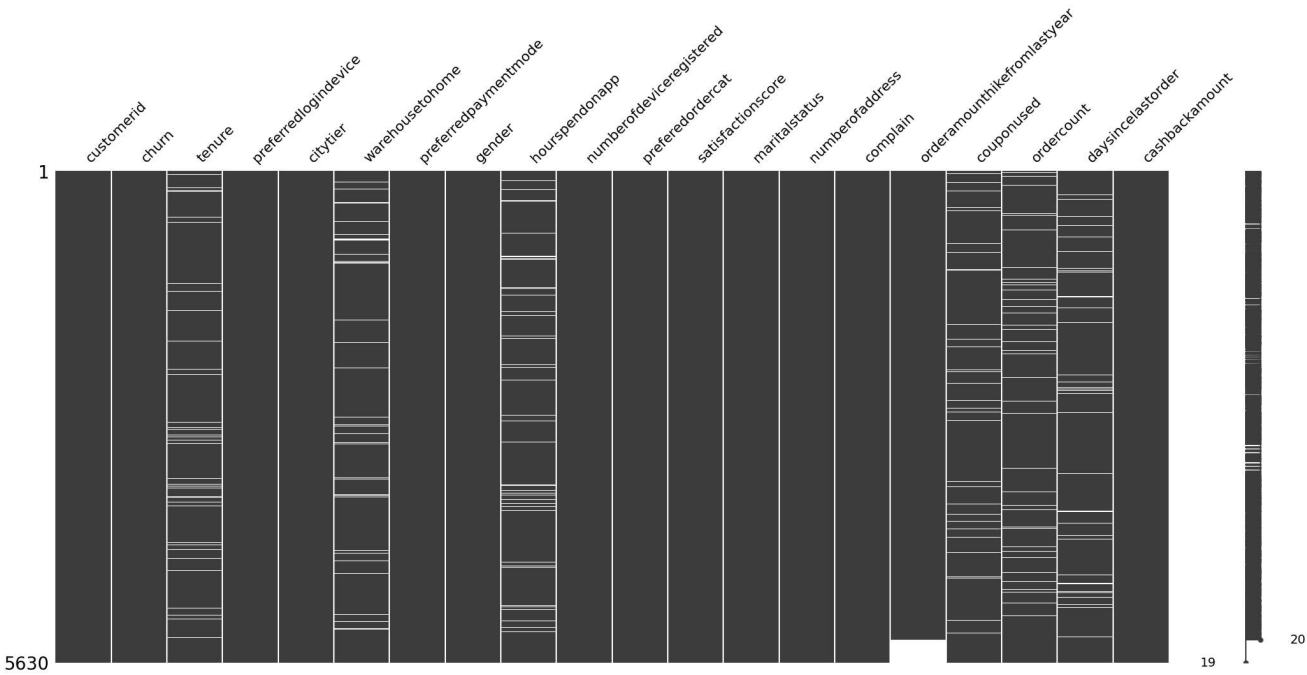
(5630, 20)
```

→ Data is not much imbalanced → Acceptable

1.1 Missing values:

```
import missingno as msno
msno.matrix(df)
```

<Axes: >



```
# Check dtypes of columns has missing values:
list_null = df.columns[df.isnull().any()].tolist()
df[list_null].dtypes
```

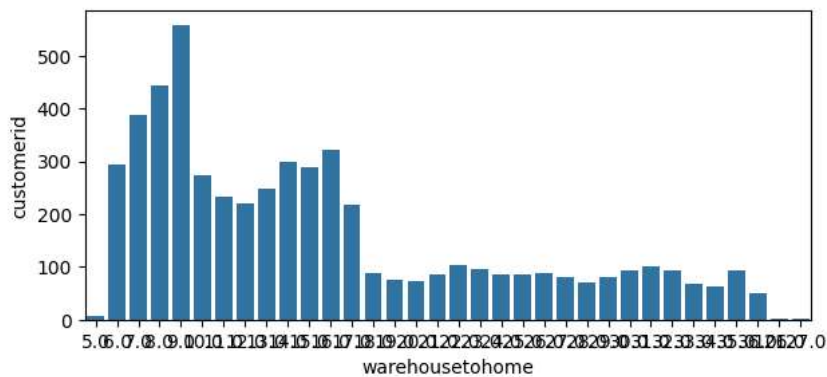
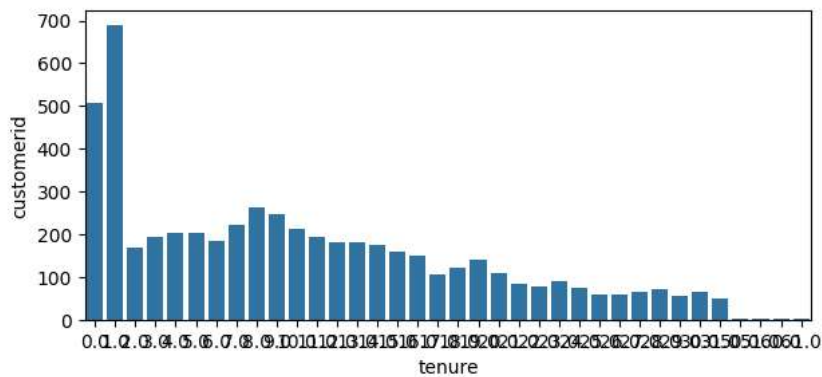
	0
tenure	float64
warehousetohome	float64
hourspendonapp	float64
orderamounthikefromlastyear	float64
couponused	float64
ordercount	float64
daysincelastorder	float64

dtype: object

```
print(df.isna().any(axis=1).mean() * 100)
```

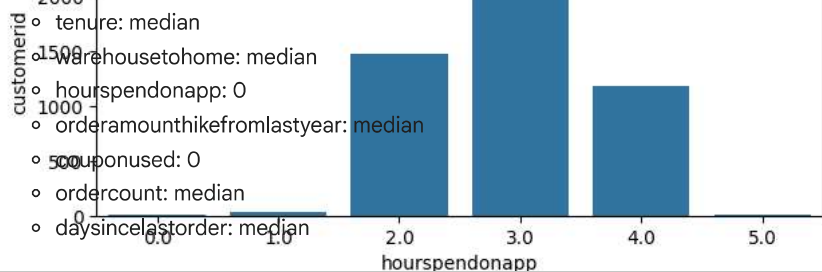
32.96625222024866

```
# Check the distribution to decide we will replace with median or 0:
for i in list_null:
    fig, ax = plt.subplots(figsize=(7, 3))
    plot_df = df.groupby(i)[['customerid']].count().reset_index()
    sns.barplot(data = plot_df, x=plot_df[i], y= plot_df['customerid'], ax=ax)
    plt.show()
```

After review the distribution, we will replace:

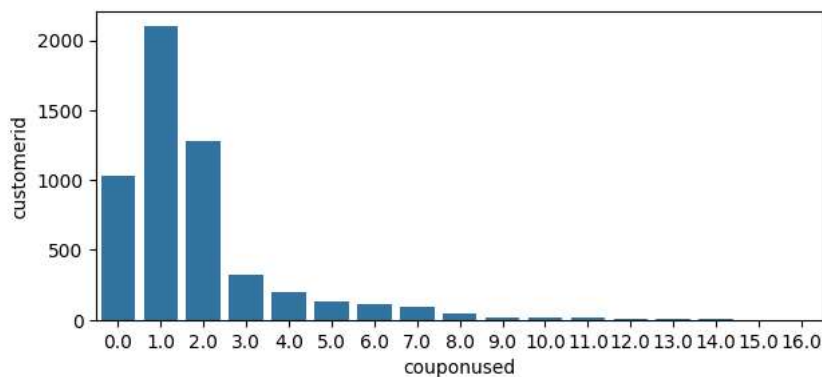
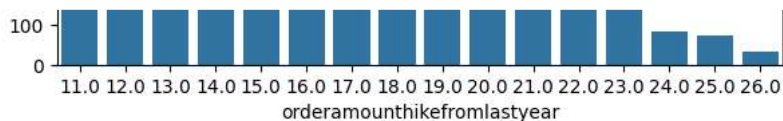
- The decision was made based on: the meaning of columns and the distribution

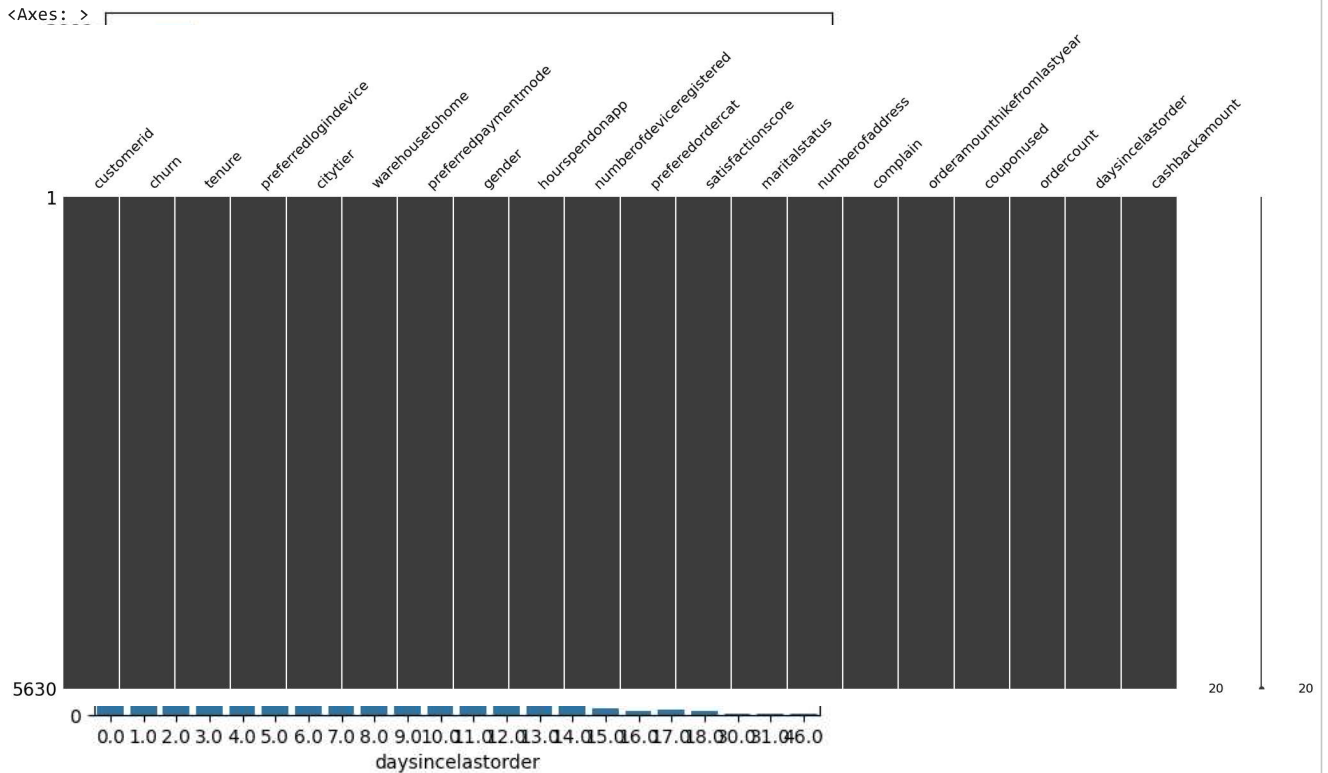


```
def replace_func(list_columns):
    for i in list_columns:
        if i in ['tenure', 'warehousetohome', 'orderamounthikefromlastyear', 'ordercount', 'daysincelastorder']:
            median = df[i].median()
            df[i].fillna(median, inplace=True)
        else:
            df[i].fillna(0, inplace=True)
```

```
replace_func(list_null)
```

```
msno.matrix(df)
```





1.2 Duplicated Values:

```
df.duplicated().any()
```

```
np.False_
```

1.3 Check same meaning values:

```
#Check the unique values:
list_obj = df.loc[:, df.dtypes == object].columns.tolist()
for j in list_obj:
    print(f"Unique values of {j}: {df[j].unique()}")
```

```
Unique values of preferredlogindevice: ['Mobile Phone' 'Computer' 'Phone']
Unique values of preferredpaymentmode: ['E wallet' 'Cash on Delivery' 'Debit Card' 'UPI' 'Credit Card' 'COD' 'CC']
Unique values of gender: ['Female' 'Male']
Unique values of preferredordercat: ['Fashion' 'Laptop & Accessory' 'Mobile Phone' 'Grocery' 'Mobile' 'Others']
Unique values of maritalstatus: ['Married' 'Single' 'Divorced']
```

```
#Replace the same meaning values:
df['preferredlogindevice'] = df['preferredlogindevice'].replace({'Mobile Phone': 'Phone'})
df['preferredpaymentmode'] = df['preferredpaymentmode'].replace({'CC': 'Credit Card', 'COD': 'Cash on Delivery'})
df['preferredordercat'] = df['preferredordercat'].replace({'Mobile Phone': 'Phone'})
```

```
#Recheck:
```

```
for j in list_obj:
    print(f"Unique values of {j}: {df[j].unique()}")
```

```
Unique values of preferredlogindevice: ['Phone' 'Computer']
Unique values of preferredpaymentmode: ['E wallet' 'Cash on Delivery' 'Debit Card' 'UPI' 'Credit Card']
Unique values of gender: ['Female' 'Male']
Unique values of preferredordercat: ['Fashion' 'Laptop & Accessory' 'Phone' 'Grocery' 'Mobile' 'Others']
Unique values of maritalstatus: ['Married' 'Single' 'Divorced']
```

Question 1:

1. Churned users usually are new users → Provide more promotion for new users, or increase the new users experience
2. Churned users usually receive less cashback than not churn → Increase the cashback ratio
3. Churned users complain more → deep dive what these churned users complain about, and provide the solution

1.3 EDA by applying Random Forest first:

```
#Encoding:
cate_columns = df.loc[:, df.dtypes == object].columns.tolist()
cate_columns
```

```
['preferredlogindevice',
 'preferredpaymentmode',
 'gender',
 'preferredordercat',
 'maritalstatus']
```

```
encoded_df = pd.get_dummies(df, columns = cate_columns, drop_first=True)
encoded_df.head(2)
```

	customerid	churn	tenure	citytier	warehousestohome	hourspondonapp	numberofdeviceregistered	satisfactionscore	numberofadvertisements
0	52869	0	20.0	3	7.0	4.0	4	5	1
1	52942	0	13.0	1	9.0	4.0	4	3	1

2 rows × 28 columns

#Applying initial Random Forest (without splitting train/test):

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import balanced_accuracy_score
```

```
x=encoded_df.drop('churn', axis = 1)
y=encoded_df[['churn']]
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.33, random_state=42)
```

```
#Normalization:
scaler = MinMaxScaler()
x_train_scaled=scaler.fit_transform(x_train)
x_test_scaled=scaler.transform(x_test)
```

```
clf_ranf = RandomForestClassifier()
clf_ranf.fit(x_train_scaled,y_train)
```

```
y_pred = clf_ranf.predict(x_test_scaled)
```

```
print(f'Balance accuracy of test set: {balanced_accuracy_score(y_test, y_pred)}')
```

Balance accuracy of test set: 0.8320094322919181

#Show Feature Importance:

```
feats = {} # a dict to hold feature_name: feature_importance
for feature, importance in zip(x.columns, clf_ranf.feature_importances_):
    feats[feature] = importance #add the name/value pair
```

```

importances = pd.DataFrame.from_dict(feats, orient='index').rename(columns={0: 'Gini-importance'})
importances = importances.sort_values(by='Gini-importance', ascending=True)

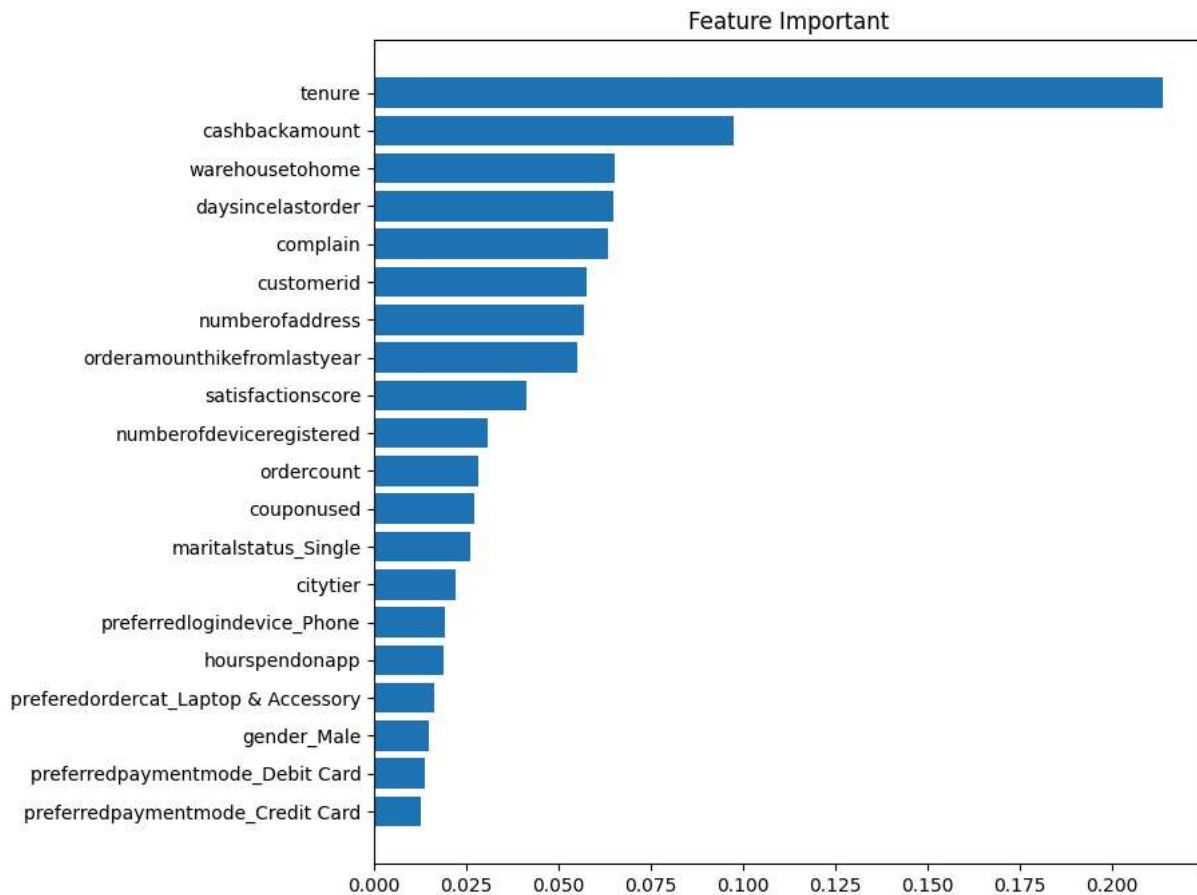
importances = importances.reset_index()

# Create bar chart
plt.figure(figsize=(8, 8))
plt.barh(importances.tail(20)['index'][:20], importances.tail(20)['Gini-importance'])

plt.title('Feature Important')

# Show plot
plt.show()

```



As Feature Importance show, we can see these features can have high relation with target columns:

- Tenure
- Cashback amount
- Distance from warehouse to home
- Complain
- Days since Last order

→ We will analyse and visualize these features for more insights.

✓ 1.4 Analyse features from initial Random Forest model:

- Tenure
- Cashback amount
- Distance from warehouse to home
- Complain
- Days since Last order

```
def count_percentage(df, column, target, count):
```

```
...
```

```
This function to create the table calculate the percentage of fraud/not fraud
```



```

this function to create the table calculate the percentage of fraud/non-fraud
transaction on total transaction group by category values
'''

# Create 2 dataframes of fraud and non-fraud
fraud = df[df[target]==1].groupby(column)[[count]].count().reset_index().sort_values(ascending=False, by = count)
not_fraud = df[df[target]==0].groupby(column)[[count]].count().reset_index().sort_values(ascending=False, by = count)

#Merge 2 dataframe into one:
cate_df = fraud.merge(not_fraud, on = column , how = 'outer')
cate_df = cate_df.fillna(0)
cate_df.rename(columns = {count+'_x':'fraud',count+'_y':'not_fraud'}, inplace = True)

#Calculate the percentage:
cate_df['%'] = cate_df['fraud']/(cate_df['fraud']+cate_df['not_fraud'])
cate_df = cate_df.sort_values(by='%', ascending=False)

return cate_df

```

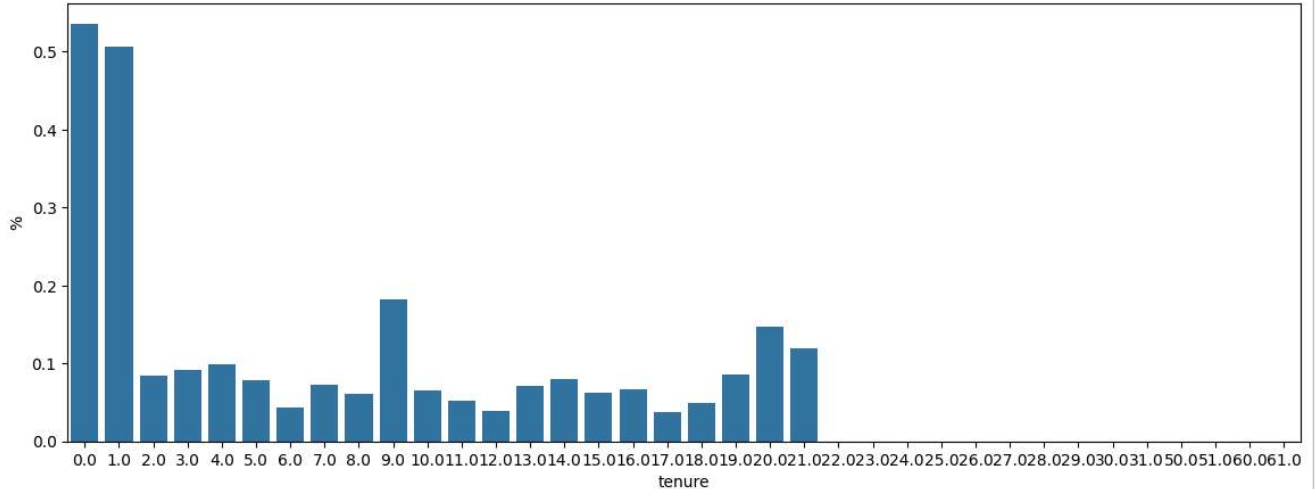
1.4.1 Tenure New users are churned more than old users (tenure = 0 or 1)

```

plot_df = count_percentage(df, 'tenure', 'churn', 'customerid')
#Visualize the data:
fig, ax = plt.subplots(figsize=(14, 5))
sns.barplot(data=plot_df, x='tenure',y='%', ax=ax)

```

<Axes: xlabel='tenure', ylabel='%>

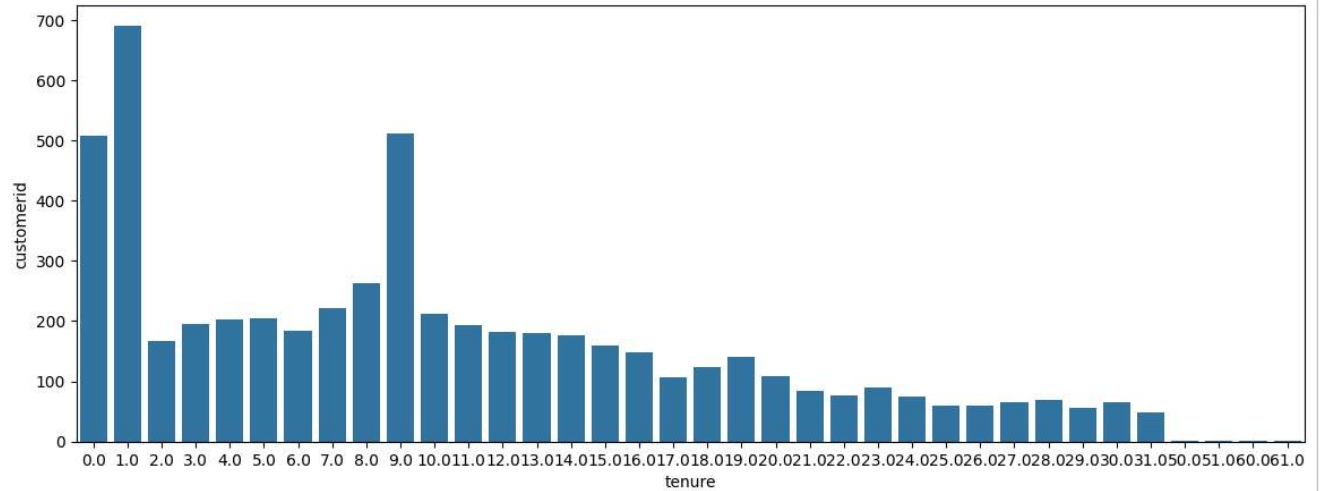


```

count_df = df.groupby('tenure')[['customerid']].count().reset_index()
#Visualize the data:
fig, ax = plt.subplots(figsize=(14, 5))
sns.barplot(data=count_df, x='tenure',y='customerid', ax=ax)

```

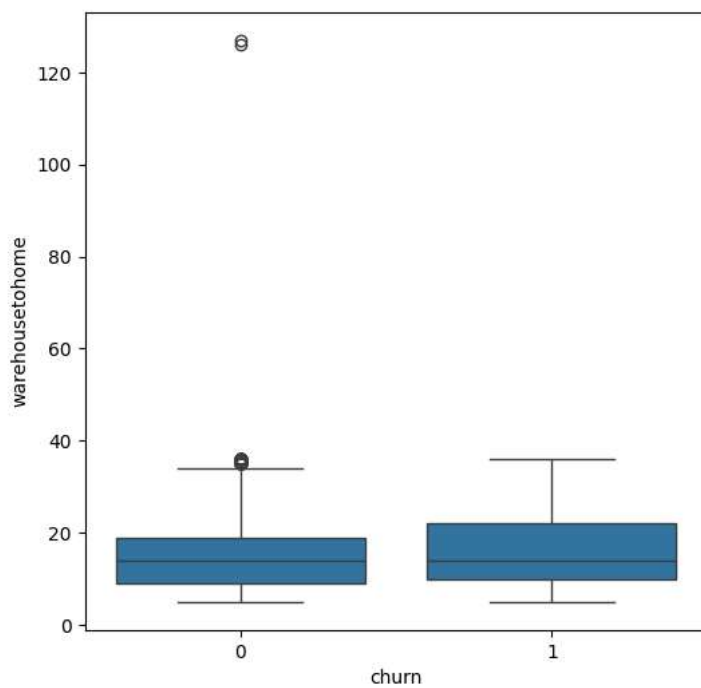
<Axes: xlabel='tenure', ylabel='customerid'>



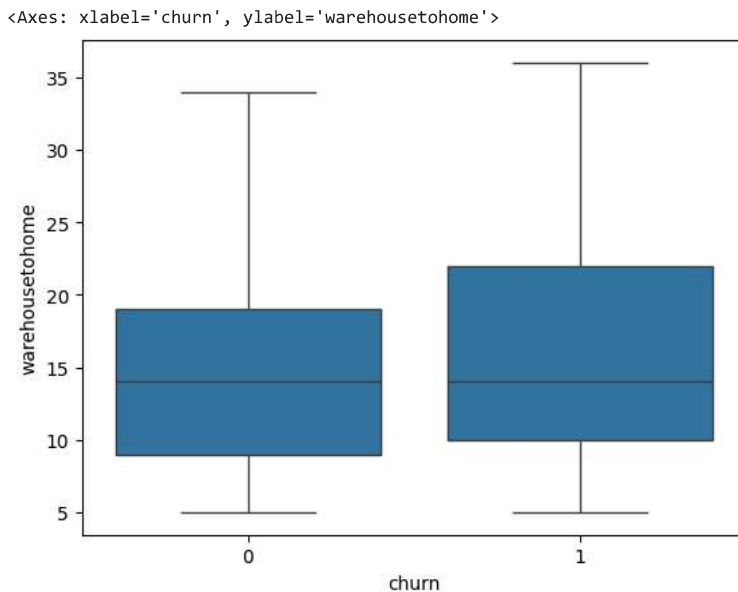
✓ 1.4.2 Warehouse to home Not significantly related

```
fig, ax = plt.subplots(figsize=(6, 6))
sns.boxplot(data=df, x='churn', y='warehousetohome', ax=ax)
```

<Axes: xlabel='churn', ylabel='warehousetohome'>



```
sns.boxplot(data=df, x='churn', y='warehousetohome', showfliers = False)
```



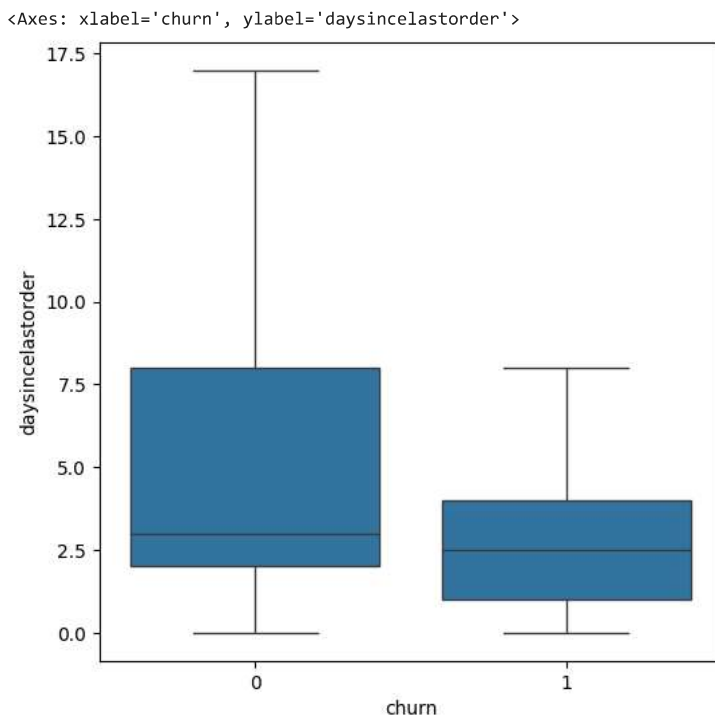
For both churn & not churn:

- The median, pt25, mean, pt75 is quite the same --> The centralize of data is the same
- For not churn, data has some outliers --> This can be not significant enough to consider it as an insight for not churn

→ There're no strong evidences show that there different between churn and not churn for warehousetohome --> We will exclude this features when apply model for not being bias.

✓ 1.4.3 Days since last order: churn users with complain = 1 have higher days since orders than churned users with complain = 0

```
fig, ax = plt.subplots(figsize=(6, 6))
sns.boxplot(data=df, x='churn', y='daysincelastorder', ax=ax, showfliers = False)
```

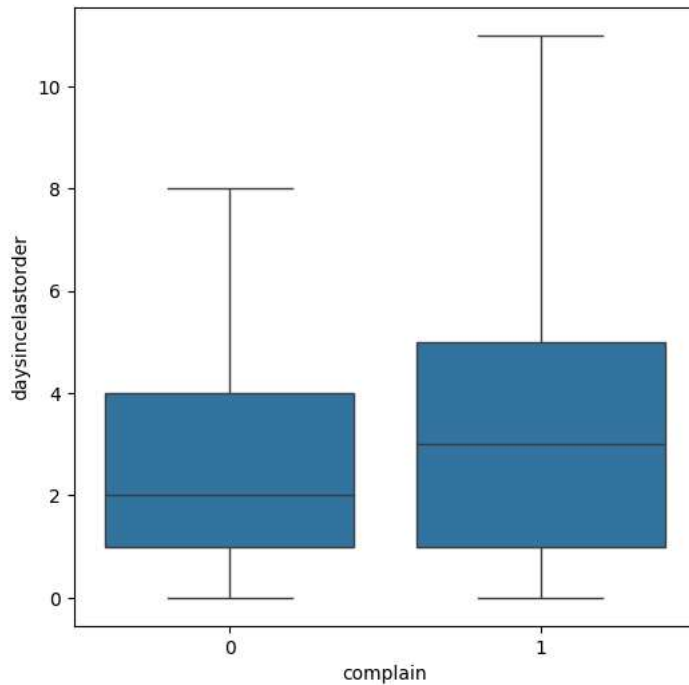


From this chart, we see for churned users, they had orders recently (the day since last order less than not churned users) --> This quite strange, we should monitor more features for this insight (satisfaction_score, complain,...)

```
churn_df = df[df['churn']==1]
fig, ax = plt.subplots(figsize=(6, 6))
```

```
sns.boxplot(data=churn_df, x='complain',y='daysincelastorder',ax=ax, showfliers = False)
```

<Axes: xlabel='complain', ylabel='daysincelastorder'>



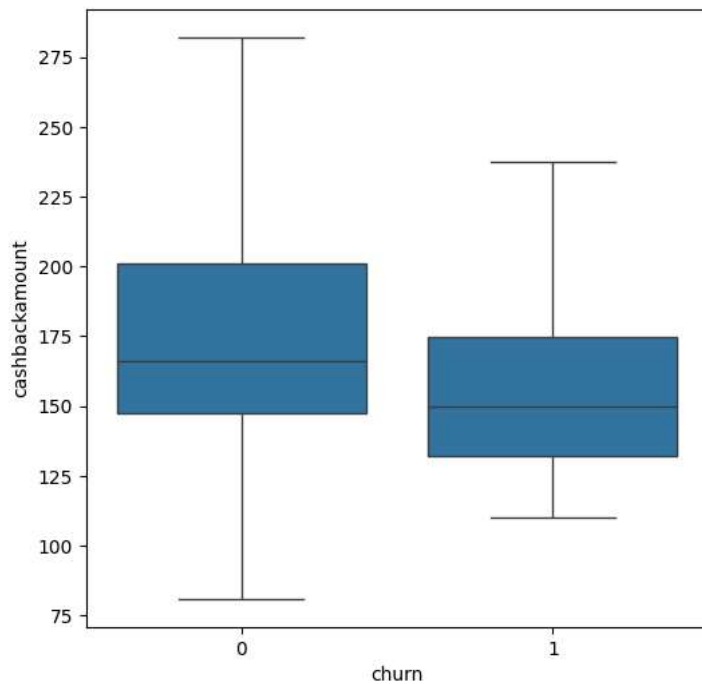
For churned users with complain = 1, they had daysincelastorder higher than churn users with complain = 0

1.4.4 Cashback amount Churn users received cashback amount less than not churn users.

```
fig, ax = plt.subplots(figsize=(6, 6))
```

```
sns.boxplot(data=df, x='churn',y='cashbackamount',ax=ax, showfliers = False)
```

<Axes: xlabel='churn', ylabel='cashbackamount'>

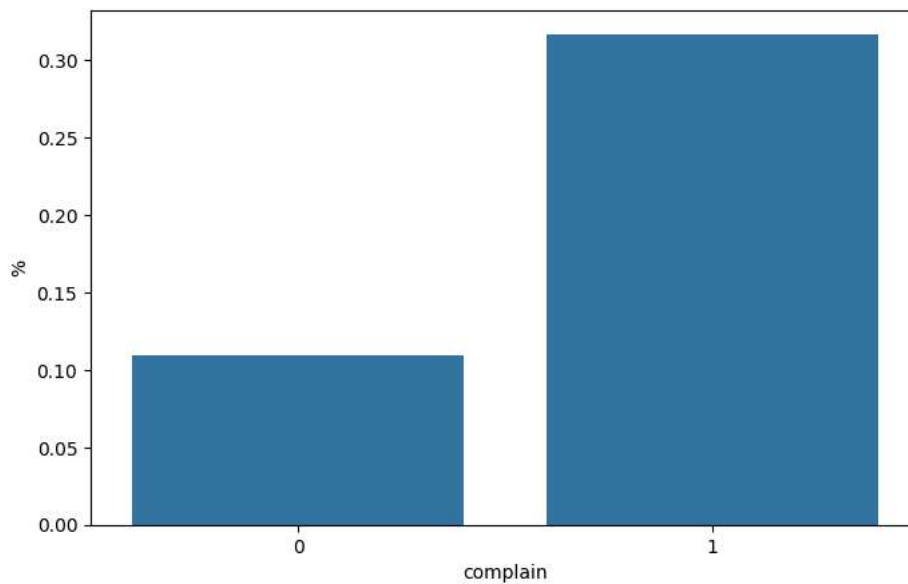


Churn users received cashback amount less than not churn users.

1.4.5 Complain The number of users complain on churn is higher than not churn

```
plot_df = count_percentage(df, 'complain', 'churn', 'customerid')
#Visualize the data:
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(data=plot_df, x='complain', y='%', ax=ax)
```

<Axes: xlabel='complain', ylabel='%>



1.4.6 Conclusion & Suggestion

1. Churned users usually are new users → Provide more promotion for new users, or increase the new users experience
2. Churned users usually receive less cashback than not churn → Increase the cashback ratio
3. Churned users complain more → deep dive what these churned users complain about, and provide the solution

Question 2:

1. Remove warehousetohome column
2. Apply Random Forest model and fine tuning

2. Feature Transforming:

```
# Remove Warehouse to home and number of address =19,20
df_model = df.drop(columns = 'warehousetohome')
df_model = df_model.drop(df_model[(df_model.numberofaddress == 19)|(df_model.numberofaddress == 20)].index)
df_model.head(1)
```

	customerid	churn	tenure	preferredlogindevice	citytier	preferredpaymentmode	gender	hourspondonapp	numberofdeviceregistered
0	52869	0	20.0	Phone	3	E wallet	Female	4.0	

```
cate_columns = df_model.loc[:, df_model.dtypes == object].columns.tolist()
encoded_df = pd.get_dummies(df_model, columns = cate_columns, drop_first=True)
encoded_df.shape
```

(5628, 27)

```
encoded_df.head(3)
```

	customerid	churn	tenure	citytier	hourspendonapp	numberofdeviceregistered	satisfactionscore	numberofaddress	complain	c
0	52869	0	20.0	3	4.0	4	5	3	0	
1	52942	0	13.0	1	4.0	4	3	2	0	
2	52972	0	16.0	3	3.0	4	3	3	0	

3 rows × 27 columns

3. Model Training & Evaluation

3.1 Split train/test set:

```
from sklearn.model_selection import train_test_split

x=encoded_df.drop(['churn','customerid'], axis = 1)
y=encoded_df[['churn']]

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size=0.3, random_state=42)

x_train, x_temp, y_train, y_temp = train_test_split(x,y, test_size=0.3, random_state=42)

x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, test_size=0.5, random_state=42)

# x_temp, x_test, y_temp, y_test = train_test_split(x,y, test_size=0.15, random_state=42)

# x_train, x_val, y_train, y_val = train_test_split(x_temp, y_temp, test_size=0.1764, random_state=42)

# Print the sizes of the splits
print(f"Training set size: {len(x_train)}")
print(f"Validation set size: {len(x_val)}")
print(f"Test set size: {len(x_test)}")
```

Training set size: 3939
Validation set size: 844
Test set size: 845

```
# scaler = MinMaxScaler()
# x_train_scaled=scaler.fit_transform(x_train)
# x_val_scaled=scaler.transform(x_val)
# x_test_scaled=scaler.transform(x_test)
```

3.2 Choose Base model:

```
models = {
    "LogisticRegression": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000))
    ]),
    "DecisionTree": DecisionTreeClassifier(max_depth=5),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=42),
    "GradientBoosting": GradientBoostingClassifier(random_state=42),
}
```

```
results = []

for name, model in models.items():
    model.fit(x_train, y_train)

    y_pred = model.predict(x_val)
    y_proba = model.predict_proba(x_val)[: , 1]

    precision = precision_score(y_val, y_pred)
    recall = recall_score(y_val, y_pred)
    bal_acc = balanced_accuracy_score(y_val, y_pred)
```

```

        results.append({
            "model": name,
            "precision": precision,
            "recall": recall,
            "balanced_accuracy": bal_acc
        })

results_df = pd.DataFrame(results).sort_values("balanced_accuracy", ascending=False)
print(results_df)

```

[Show hidden output](#)

3.3 Enhance Random Forest model

```

from sklearn.model_selection import GridSearchCV
# Define the parameter grid

clf_rand = RandomForestClassifier(random_state=0)

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [20, 30, 40],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

# Use GridSearchCV to find the best parameters
grid_search = GridSearchCV(clf_rand, param_grid, cv=5, scoring='balanced_accuracy')

# Fit the model
grid_search.fit(x_train, y_train)

# Print the best parameters
print("Best Parameters: ", grid_search.best_params_)

# Evaluate the best model on the test set
best_clf = grid_search.best_estimator_
print(best_clf)

Best Parameters: {'bootstrap': False, 'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 100}
RandomForestClassifier(bootstrap=False, max_depth=20, random_state=0)

```

Re-apply model with new parameters:

```

best_params = grid_search.best_params_
clf_rand_after = RandomForestClassifier(**best_params, random_state=0)

clf_rand_after.fit(x_train_scaled, y_train)
y_ranf_aft_train = clf_rand_after.predict(x_train_scaled)
y_ranf_aft_val = clf_rand_after.predict(x_val_scaled)

```

```

print(f'Balance accuracy of train set: {balanced_accuracy_score(y_train, y_ranf_aft_train)}')
print(f'Balance accuracy of val set: {balanced_accuracy_score(y_val, y_ranf_aft_val)}')

```

```

Balance accuracy of train set: 1.0
Balance accuracy of val set: 0.9003660063262713

```

→ The balanced accuracy increase to 90 → Can use this model as final model

```

y_pred_test = clf_rand_after.predict(x_test_scaled)
print(f'Balance accuracy of test set: {balanced_accuracy_score(y_test, y_pred_test)}')

```

```

Balance accuracy of test set: 0.8854548229548229

```

When reapply model on test set, the accuracy is 88%, acceptable

Question 3:

- Use K-Means to clustering churn-users groups.
- Find the insight between the groups

1. Get the data prepared

We will get all features of churned users for clustering

```
#Prepare data:
df_churned = df[df['churn']==1]
df_churned.drop(columns = ['customerid', 'churn'],inplace=True)
print(df_churned.shape)
df_churned.head(2)
```

(948, 18)

	tenure	preferredlogindevice	citytier	warehousetohome	preferredpaymentmode	gender	hourspondonapp	numberofdeviceregiste
7	1.0	Computer	1	17.0	Credit Card	Female	4.0	
24	1.0	Computer	1	17.0	Credit Card	Female	4.0	

```
#Transform data:
encoded_df = pd.get_dummies(df_churned, columns = cate_columns,drop_first=True)

#Normalization:
scaler = MinMaxScaler()
model=scaler.fit(encoded_df)
scaled_data=model.transform(encoded_df)
scaled_df = pd.DataFrame(scaled_data, columns = encoded_df.columns.tolist())
```

```
scaled_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 948 entries, 0 to 947
Data columns (total 26 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   tenure                                948 non-null    float64
1   citytier                              948 non-null    float64
2   warehousetohome                       948 non-null    float64
3   hourspondonapp                        948 non-null    float64
4   numberofdeviceregistered              948 non-null    float64
```